

DEVOPS MASTER COURSE

Volume 9E

Integrated Terraform-Ansible Delivery

GitOps, Security, Governance, Compliance, Observability, and Enterprise Automation

Chapters 41-55

Production objective: build an auditable delivery system in which Terraform provisions infrastructure, Ansible configures and validates it, policy controls risk, and Git records the intended state and evidence of every change.

2026 Edition

About Volume 9E

Part E unifies the two automation disciplines developed earlier in Volume 9. Terraform remains responsible for durable infrastructure resource lifecycle, dependency graphs, provider interactions, and state. Ansible remains responsible for operating-system, middleware, application, network, and post-provision configuration. The engineering challenge is not merely invoking one tool after another; it is designing stable handoff contracts, approval boundaries, evidence, recovery paths, and ownership across the entire delivery system.

The chapters use vendor-neutral patterns that can be implemented in GitHub Actions, GitLab CI/CD, Jenkins, Azure DevOps, HCP Terraform, AWX, Ansible Automation Platform, or another governed automation service. Examples are intentionally portable. Product-specific capabilities are discussed only where they illustrate a broader operating principle.

Core principle: Terraform and Ansible should cooperate through explicit, versioned interfaces. They should not compete for ownership of the same mutable object or depend on unreviewed shell glue.

Volume 9 Structure and Progress

Part	Primary Focus	Status
9A	Terraform foundations, language, state, modules, import, and team workflow	Complete
9B	Advanced Terraform testing, policy, HCP operations, security, and Stacks	Complete
9C	Ansible foundations, inventory, playbooks, variables, templates, and Vault	Complete
9D	Advanced Ansible roles, collections, controller, Windows, networks, testing, and scale	Complete
9E	Integrated delivery, GitOps, security, governance, observability, and compliance	This book
9F	Enterprise projects, troubleshooting, certification, interviews, and final capstone	Next

Part E Chapter Map

Chapter	Title
41	Terraform-Ansible Operating Model and Ownership Boundaries
42	Repository Architecture, Environments, and GitOps Principles
43	Terraform-to-Ansible Handoff Contracts and Dynamic Inventory

Chapter	Title
44	Continuous Integration for Infrastructure and Configuration Code
45	Continuous Delivery Orchestration and Progressive Infrastructure Change
46	Workload Identity, Secrets, and Keyless Automation
47	Policy as Code, Compliance Gates, and Exception Management
48	Software Supply-Chain Security, Provenance, and Artifact Trust
49	Integrated Testing, Ephemeral Environments, and Release Qualification
50	Drift Detection, Reconciliation, and GitOps Operations
51	Automation Observability, SLOs, and Evidence Engineering
52	Change Risk, Approvals, Rollback, and Incident-Safe Delivery
53	Multi-Cloud Platform Factories and Self-Service Automation
54	Audit, Compliance Evidence, Control Mapping, and Governance
55	Enterprise Integrated Automation Platform Capstone

Enterprise Lab Safety and Evidence Standard

- Run all labs in disposable accounts, subscriptions, projects, local virtual machines, containers, or approved sandboxes. Apply quotas, budgets, TTL labels, and automatic cleanup.
- Use protected branches, peer review, signed commits or equivalent identity evidence, pinned toolchains, and dependency lock files. Never execute unreviewed plans or playbooks against production.
- Use short-lived workload identity whenever the CI/CD platform and target system support federation. Avoid static cloud keys, personal access tokens, embedded SSH keys, and plaintext Vault passwords.
- Separate plan, approval, apply, configuration, verification, and promotion evidence. A successful pipeline is not sufficient proof unless the exact artifacts and target environment can be reconstructed.
- Define a recovery decision before every destructive test: rollback, roll-forward, restore, recreate, or manual intervention. Record the operator, trigger, and evidence required for each path.
- Do not let Terraform and Ansible manage the same property. Ownership conflicts create perpetual drift, non-idempotent changes, and unsafe recovery behavior.
- Scrub state, plan JSON, Ansible output, controller events, logs, and screenshots before sharing them. Sensitive values can leak through diagnostics even when marked sensitive.

Chapter 41: Terraform-Ansible Operating Model and Ownership Boundaries

An integrated automation platform succeeds only when each tool has a clear responsibility. Terraform is strongest when managing declarative resource lifecycle through provider APIs and durable state. Ansible is strongest when converging machines, services, applications, and network devices toward a desired runtime configuration. This chapter defines the boundary, the handoff, and the operational contract between them.

Learning Objectives

- Assign infrastructure and configuration responsibilities without overlap.
- Design explicit lifecycle and data contracts between Terraform and Ansible.
- Choose push, pull, event-driven, or controller-mediated execution patterns.
- Define ownership, support, and recovery responsibilities for each automation stage.

Core Concepts

Resource ownership: Every mutable attribute has one authoritative automation owner. Terraform may create a virtual machine and its security group, while Ansible owns packages, files, users, services, and application configuration inside that machine. If both tools edit a firewall rule, tag, file, or service property, drift and rollback behavior become ambiguous.

Provision-configure-verify lifecycle: A safe workflow separates infrastructure provisioning, configuration convergence, and independent verification. The handoff should carry stable identifiers, addresses, roles, environment metadata, and versioned outputs rather than raw state files or ad hoc parsing.

Execution topology: Small teams may run Terraform and Ansible sequentially in one pipeline. Larger platforms often use separate workspaces and controller jobs connected by events, APIs, or approved run triggers. Decoupling improves ownership and retries but requires explicit correlation identifiers and failure semantics.

Failure domains: Terraform apply failure, inventory generation failure, Ansible convergence failure, and verification failure are different incidents. Each stage needs its own retry policy and recovery path. Blindly rerunning an entire pipeline can duplicate external actions or conceal partial success.

Control-plane separation: Credentials, state, inventory, execution environments, logs, approvals, and policy engines are control-plane assets. They require stronger access controls and retention than ordinary application data because they can modify many systems.

Operational contract: The contract defines input schema, output schema, expected idempotency, target selection, timeout, concurrency, audit evidence, rollback mechanism, and support owner. Treat the contract as a versioned API between platform components.

Reference Configuration or Command Pattern

```
# pipeline contract (illustrative YAML)
change_id: CHG-2026-0042
environment: staging
terraform:
  workspace: payments-staging
  plan_digest: sha256:...
  expected_outputs:
    - host_ids
    - private_addresses
    - service_role
ansible:
  job_template: configure-payments
  inventory_contract: v2
  limit: payments_app
verification:
  checks: [health, policy, telemetry]
  rollback: restore_previous_release
```

Production Checklist

- A responsibility matrix identifies one owner for each resource and property.
- Terraform outputs expose only stable, necessary handoff data.
- Ansible inventory and variables are generated from approved sources, not direct state-file scraping in untrusted jobs.
- Each stage has a unique correlation ID and retained evidence.
- Partial-success recovery is tested before production use.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Terraform and Ansible repeatedly undo each other	Overlapping ownership of a mutable property	Remove one owner, import or reconcile the current value, and document the new boundary	Before/after plan, task diff, ownership decision record
Provisioning succeeds but configuration targets the wrong hosts	Unstable or stale inventory handoff	Use immutable instance IDs and tagged grouping; refresh inventory after apply	Output artifact, inventory graph, target limit
Pipeline rerun creates duplicate side effects	Non-idempotent external actions in hooks	Add idempotency keys, stage checkpoints, and resume logic	Run ID, action response, replay test
No team accepts a failed stage	Unclear service ownership	Define RACI, escalation, and handoff SLA for every stage	Service catalog entry, on-call routing

Hands-on Lab 41: Define and Prove an Automation Ownership Contract

Prerequisites

- A small Terraform project that creates one or more disposable hosts.
- An Ansible playbook that configures a service on those hosts.
- A Git repository for architecture records and evidence.

Implementation

1. Create a matrix listing every resource and property changed by Terraform or Ansible.
2. Identify conflicts and assign a single authoritative owner for each property.

3. Define a versioned JSON or YAML handoff schema containing host IDs, addresses, groups, environment, and change ID.
4. Modify Terraform outputs and the Ansible inventory process to use only the approved schema.
5. Inject failures after apply and midway through configuration; document safe resume and recovery actions.
6. Run the workflow twice and verify that the second execution is idempotent.

Validation Evidence

- No mutable property has dual ownership.
- The generated handoff artifact validates against the documented schema.
- A failed configuration run can resume without repeating infrastructure creation.
- Second execution produces no unintended changes.

Cleanup

- Destroy the disposable infrastructure and revoke temporary credentials.
- Retain the sanitized contract, responsibility matrix, and failure evidence.

Interview Questions and Model Answers

1. Why should Terraform and Ansible not manage the same property?

Their state and convergence models are independent. Dual ownership produces perpetual drift, unpredictable ordering, and unclear rollback responsibility.

2. What belongs in the handoff contract?

Stable identifiers, target grouping, environment metadata, version, correlation ID, expected outputs, and recovery semantics - not unrestricted state or secrets.

3. When is a decoupled controller workflow preferable?

When teams, credentials, retries, approvals, or scaling differ between provisioning and configuration, and the organization can support explicit event and evidence contracts.

4. What is the most important test for the integration?

A controlled partial failure followed by a safe resume, because happy-path sequential execution does not prove recoverability.

Chapter Summary

- Terraform owns durable infrastructure lifecycle; Ansible owns runtime convergence.
- Ownership boundaries must be explicit and versioned.
- Partial failure is normal and must be designed into the workflow.
- The handoff is an API contract, not incidental pipeline glue.

Chapter 42: Repository Architecture, Environments, and GitOps Principles

Git is more than a file store for automation. It is the review, traceability, promotion, and recovery system for desired state. A scalable repository design separates reusable products from environment composition while preserving a clear path from a reviewed commit to the running system.

Learning Objectives

- Design repositories for modules, roles, environments, policies, and pipelines.
- Apply vendor-neutral GitOps principles to infrastructure and configuration.
- Choose trunk, release, or environment promotion models.
- Protect repositories as production control-plane systems.

Core Concepts

Declarative desired state: GitOps starts with a declarative description of the target system. Terraform configuration, Ansible roles, inventory composition, policy, and release metadata should describe outcomes rather than undocumented operator sequences.

Versioned and immutable history: Every production change should map to a reviewed commit, signed tag, or immutable artifact. Force pushes, mutable release tags, and untracked controller edits weaken recovery and auditability.

Pulled automatically: A reconciler or controlled automation service should obtain desired state from the approved source rather than accepting arbitrary operator payloads. In push-based infrastructure pipelines, the same principle is approximated by protected workflow definitions and immutable run artifacts.

Continuously reconciled: GitOps is incomplete if drift is detected but not classified and resolved. Reconciliation can be automatic for low-risk changes and approval-gated for destructive or regulated changes.

Repository topology: Separate reusable modules and collections from live environment composition when release cadence and ownership differ. Monorepositories simplify atomic changes; multiple repositories improve isolation. The decision should be driven by ownership and promotion requirements, not fashion.

Environment strategy: Long-lived branches often diverge and hide unmerged fixes. Prefer one source version promoted through environment-specific variables, deployment records, or release manifests unless regulatory controls require a different model.

Reference Configuration or Command Pattern

```
automation-platform/
modules/           # versioned Terraform modules
collections/       # versioned Ansible collections
live/
  dev/
  staging/
  production/
policies/
  terraform/
  ansible/
pipelines/
schemas/
docs/adr/
evidence/.gitkeep
```

Production Checklist

- Protected branches require reviewed and passing changes.

- Release tags and published automation artifacts are immutable.
- Production environments consume versioned modules, collections, and execution images.
- Controller-side edits are disabled or reconciled back to Git.
- Repository permissions implement least privilege and separation of duties.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Production differs from the repository	Manual controller or console changes	Disable unmanaged edits, detect drift, and reconcile through reviewed code	Drift report, actor logs, corrective commit
Environment branches cannot be merged	Long-lived branch divergence	Move environment differences to data and promote one release lineage	Merge graph, release manifest
A module fix changes all environments unexpectedly	Floating version reference	Pin versions and promote updates explicitly	Lock file, module digest, promotion record
Pipeline definition can be changed by deployers	Insufficient repository separation of duties	Protect workflow paths and require platform-owner review	CODEOWNERS, branch rule export

Hands-on Lab 42: Build a GitOps-Oriented Repository and Promotion Model

Prerequisites

- A Git hosting service or local repository with branch protection simulation.
- Sample Terraform module and Ansible role or collection.
- Three disposable environment directories.

Implementation

1. Create a repository structure separating reusable content, live composition, policy, schemas, and pipeline definitions.
2. Define CODEOWNERS or equivalent approval rules for modules, Ansible content, policies, and production environments.
3. Publish a versioned module and collection artifact; pin them in the development environment.
4. Promote the same versions to staging and production through release manifests rather than branch copying.
5. Introduce an out-of-band change, detect it, and reconcile through a reviewed commit.
6. Document the chosen GitOps model, including where continuous reconciliation is automatic and where it requires approval.

Validation Evidence

- Every environment references immutable content versions.
- A production change requires both technical checks and authorized review.
- The out-of-band change is linked to a drift record and corrective commit.
- The repository can reconstruct the intended production configuration.

Cleanup

- Remove disposable environments and revoke test tokens.
- Retain the repository topology ADR and promotion evidence.

Interview Questions and Model Answers

1. What are the core GitOps characteristics?

Desired state is declarative, versioned and immutable, applied through controlled automation, and continuously reconciled against the running system.

2. Why are environment branches risky?

They accumulate divergence, duplicate fixes, and make it difficult to prove which release is actually promoted across environments.

3. When is a monorepository appropriate?

When atomic cross-component changes and shared ownership outweigh the need for strict isolation and independent release cadence.

4. Why treat the Git repository as production infrastructure?

Compromise of desired state or workflow definitions can change every managed environment, so repository security is part of the automation control plane.

Chapter Summary

- Git is the review and recovery backbone for automation.
- Immutable versions make promotion and rollback reproducible.
- Environment differences should be data, not uncontrolled code forks.
- GitOps requires reconciliation, not only Git-based deployment.

Chapter 43: Terraform-to-Ansible Handoff Contracts and Dynamic Inventory

The handoff from provisioning to configuration is one of the most failure-prone parts of an automation platform. Reliable systems avoid fragile text parsing and unrestricted access to state. They expose a small, validated contract and build inventory from authoritative APIs, approved outputs, or controller-managed sources.

Learning Objectives

- Design stable Terraform outputs for downstream automation.
- Generate dynamic Ansible inventory safely.
- Map infrastructure metadata into groups and host variables.
- Handle resource replacement, eventual consistency, and inventory caching.

Core Concepts

Output design: Outputs should represent a consumer-facing interface: IDs, addresses, roles, zones, and versioned metadata. Avoid exporting full resource objects, credentials, or provider-specific internals that create tight coupling.

Inventory sources: Preferred sources are cloud inventory plugins, controller inventory sources, or a purpose-built service that queries authoritative APIs. The `cloud.terraform terraform_state` plugin can be useful in controlled environments, but state access must be tightly limited because state can contain sensitive data.

Grouping rules: Derive groups from stable labels, tags, resource roles, and environment metadata. Avoid grouping only by IP address or human-readable name when replacement changes those values.

Eventual consistency: A newly created resource may not immediately accept SSH, appear in inventory APIs, receive DNS, or complete bootstrap. The handoff needs readiness checks and bounded retries rather than arbitrary sleep commands.

Caching and freshness: Dynamic inventory caching improves performance but can retain deleted or replaced hosts. Cache TTL, invalidation after apply, and evidence of the resolved inventory are required for safe targeting.

Schema versioning: Inventory payloads should include a version and validate required fields. Consumers must reject incompatible versions rather than silently guessing.

Reference Configuration or Command Pattern

```
# Terraform output: stable consumer interface
output "ansible_inventory" {
  value = {
    schema_version = "2"
    hosts = {
      for name, vm in module.compute.instances : name => {
        id      = vm.id
        address = vm.private_ip
        groups  = [vm.role, var.environment]
      }
    }
  }
  sensitive = false
}

# Pipeline
terraform output -json ansible_inventory > inventory-contract.json
python validate_inventory.py inventory-contract.json
ansible-inventory -i inventory_from_contract.py --graph
```

Production Checklist

- Outputs expose the minimum necessary data and have a documented schema.
- Inventory is refreshed or invalidated after infrastructure changes.
- Target groups are built from stable metadata.
- Connectivity readiness is verified before configuration begins.
- Resolved inventory is retained with the change evidence.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Ansible targets deleted hosts	Stale inventory cache	Invalidate cache after apply and shorten TTL for volatile fleets	Cache metadata, inventory graph
New host is unreachable	Provisioning completed before bootstrap or route readiness	Use bounded readiness probes with diagnostics	Bootstrap log, route test, retry timeline
Inventory job exposes secrets	Direct unrestricted state access	Use a reduced output contract or API inventory source	State access audit, redacted contract
Host changes groups after replacement	Grouping based on unstable name or address	Use immutable role and environment metadata	Old/new inventory diff

Hands-on Lab 43: Create a Versioned Dynamic Inventory Contract

Prerequisites

- Terraform configuration that creates two or more disposable hosts.
- Ansible installed with access to the targets.
- A JSON schema validator or equivalent validation script.

Implementation

1. Define a minimal versioned inventory output in Terraform.
2. Create a schema that validates host ID, address, groups, environment, and optional connection metadata.
3. Generate an Ansible inventory source from the output without exposing the full state file.

4. Add readiness checks for network path, SSH, and Python availability.
5. Replace one host and prove that stale inventory is invalidated before the next Ansible run.
6. Retain the exact resolved inventory graph and configuration result for the change.

Validation Evidence

- The contract fails closed when required fields or schema version are invalid.
- No sensitive state values appear in the inventory artifact.
- Replacement targets only the current host set.
- Ansible reaches all intended hosts and no unintended host.

Cleanup

- Destroy the disposable hosts and clear inventory caches.
- Retain the schema, sanitized inventory, and replacement test evidence.

Interview Questions and Model Answers

1. Why not give Ansible unrestricted access to Terraform state?

State may contain sensitive values and provider internals, and it tightly couples the configuration layer to Terraform implementation details.

2. What makes a good Terraform output contract?

It is minimal, stable, documented, versioned, validated, and based on consumer needs rather than raw resource structure.

3. How do you prevent stale inventory?

Invalidate after infrastructure changes, use appropriate TTLs, and retain the resolved inventory used by each run.

4. Why are readiness checks better than fixed sleeps?

They verify the actual dependency and provide diagnostic evidence while avoiding both unnecessary delay and premature execution.

Chapter Summary

- The handoff should be a small, validated API.
- Dynamic inventory must balance freshness, performance, and least privilege.
- Stable metadata is essential for correct grouping.
- Readiness and replacement tests prove the integration.

Chapter 44: Continuous Integration for Infrastructure and Configuration Code

Continuous integration for automation must evaluate syntax, semantics, behavior, policy, and compatibility without changing production. A mature CI system creates reproducible tool environments, caches safely, runs tests at the correct layer, and publishes immutable artifacts and machine-readable evidence.

Learning Objectives

- Design layered CI checks for Terraform and Ansible.
- Build reproducible toolchain and dependency environments.
- Generate and inspect machine-readable plan and test artifacts.
- Parallelize safely while preserving deterministic results.

Core Concepts

Fast feedback layers: Formatting, syntax, linting, schema validation, and static policy should run first. Unit and contract tests follow. Provider-backed plans, Molecule scenarios, and ephemeral integration tests run only after cheaper checks pass.

Reproducible toolchain: Pin Terraform, providers, ansible-core, collections, Python dependencies, lint rules, policy bundles, and execution images. The pipeline should record checksums and image digests so a result can be reproduced.

Terraform CI: A typical sequence is fmt check, init with a controlled plugin cache, validate, test, plan, show -json, policy evaluation, cost or risk analysis, and artifact publication. Do not apply from pull-request jobs.

Ansible CI: Use syntax-check, ansible-lint, collection build checks, role argument validation, Molecule or equivalent scenarios, idempotency tests, and platform compatibility matrices. Test from the built artifact, not only the source tree.

Artifact discipline: Publish the saved Terraform plan, plan JSON, lock file, module manifest, Ansible collection artifact, execution image digest, test reports, and policy results. Apply should consume the approved artifacts rather than recomputing them.

Untrusted contributions: Fork and pull-request jobs must not receive production secrets or privileged runners. Provider-backed checks should use isolated credentials with strict quotas or mocked providers.

Reference Configuration or Command Pattern

```
stages:
- static:
  - terraform fmt -check -recursive
  - terraform validate
  - ansible-lint
  - ansible-playbook --syntax-check site.yml
- test:
  - terraform test
  - molecule test
- plan:
  - terraform plan -out=tfplan -input=false
  - terraform show -json tfplan > tfplan.json
  - opa eval --data policy/ --input tfplan.json
- package:
  - ansible-galaxy collection build
  - cosign attest ...
```

Production Checklist

- All tool and dependency versions are pinned and recorded.
- Pull-request jobs run without production credentials.
- Policy evaluates machine-readable artifacts, not human text.
- Apply jobs consume approved plan and content artifacts.
- CI failures identify the responsible layer and preserve diagnostics.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Plan differs between CI runs	Unpinned providers, modules, or external data	Pin dependencies and stabilize data sources	Lock files, dependency manifests
Molecule passes locally but fails in CI	Different execution environment or container runtime	Build and test with the same execution image	Image digest, environment report
Policy parser breaks on CLI text	Human-readable output used as input	Use terraform show -json and structured reports	Plan JSON, policy command
PR can access production secrets	Overprivileged runner or secret exposure rule	Separate trusted and untrusted workflows	Runner policy, secret access log

Hands-on Lab 44: Build a Layered CI Pipeline for Terraform and Ansible

Prerequisites

- The Terraform module and Ansible collection from earlier parts.
- A CI runner or local pipeline simulator.
- OPA or another policy engine and a container runtime.

Implementation

1. Create a pinned toolchain manifest and dependency lock files.
2. Implement fast formatting, validation, lint, and syntax stages.
3. Add Terraform tests and an Ansible Molecule idempotency scenario.
4. Generate a saved plan and plan JSON in an isolated test environment.
5. Evaluate policy and publish all approved artifacts with checksums.
6. Simulate an untrusted pull request and verify that no privileged credentials are available.

Validation Evidence

- The same commit produces identical dependency manifests and artifact digests.
- A policy violation fails before any apply capability exists.
- Molecule proves changed first run and unchanged second run.
- Untrusted jobs cannot access production secrets or runners.

Cleanup

- Delete ephemeral test resources and caches containing credentials.
- Retain sanitized reports, manifests, and artifact checksums.

Interview Questions and Model Answers

1. Why save the Terraform plan artifact?

It allows approval and apply to reference the exact reviewed change instead of recalculating against a potentially different state or configuration.

2. What should run first in CI?

Fast deterministic checks such as formatting, syntax, linting, validation, and schema policy, because they reduce expensive downstream work.

3. Why test Ansible from a built collection or execution image?

It proves the distributable artifact and dependency set, not only the developer source tree.

4. How should forked pull requests be handled?

Run them without privileged secrets and use mocks, disposable credentials, or approval-gated trusted workflows for provider-backed tests.

Chapter Summary

- CI is evidence generation, not deployment.
- Structured artifacts enable policy and audit.
- Pinned toolchains make results reproducible.
- Untrusted changes require isolated execution.

Chapter 45: Continuous Delivery Orchestration and Progressive Infrastructure Change

Continuous delivery coordinates approved artifacts into controlled changes. Infrastructure delivery differs from stateless application deployment because resources may be durable, shared, stateful, or impossible to roll back automatically. The pipeline must preserve the exact plan, sequence dependencies, configure targets, verify outcomes, and stop safely on uncertainty.

Learning Objectives

- Design plan-approval-apply-configure-verify workflows.
- Use progressive delivery for infrastructure and configuration.
- Separate automatic rollback from operator-driven recovery.
- Implement promotion and environment protection controls.

Core Concepts

Artifact promotion: Promote the exact Terraform plan where safe, or at minimum the exact configuration commit, module versions, lock file, and policy bundle. Promote Ansible collection and execution image digests rather than rebuilding per environment.

Approval context: Approvers need a concise change summary, risk classification, policy results, cost impact, blast radius, maintenance window, rollback decision, and verification plan. An approval button without context is not governance.

Progressive infrastructure change: Use canary instances, rolling replacement, small target subsets, serial Ansible batches, or separate deployment groups. Progress based on health and policy evidence, not elapsed time alone.

Apply then configure: After Terraform apply, refresh authoritative inventory and wait for readiness. Run Ansible with a bounded target limit, serial strategy, and explicit failure threshold. Independent verification should run after convergence.

Rollback limits: Some infrastructure changes cannot be reverted safely. The recovery action may be restore, replacement, failover, or roll-forward. The pipeline should not promise rollback unless it has been tested for that change class.

Environment protection: Production applies should run only from protected environments, trusted runners or agents, approved identities, and immutable workflow definitions. Break-glass paths must be logged and reviewed.

Reference Configuration or Command Pattern

```
delivery:
  plan:
    artifact: tfplan
    approvals: [platform, service_owner]
  apply:
    command: terraform apply -input=false tfplan
  inventory:
    refresh: true
    readiness_timeout: 10m
  configure:
    command: ansible-playbook site.yml --limit canary --serial 1
  verify:
    gates: [health, logs, policy, business_smoke]
  promote:
    next_batch: 25%
    stop_on_error_rate: 1%
```

Production Checklist

- Approval references immutable artifacts and complete risk context.
- The apply identity cannot modify workflow definitions.
- Progressive batches have measurable promotion and abort criteria.
- Configuration uses refreshed inventory and explicit limits.
- Recovery paths are change-class specific and tested.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Approved plan cannot be applied	State changed after approval	Expire the plan and require re-plan/review	State serial, old/new plan digest
Canary passes but large batch fails	Capacity or concurrency effect	Reduce batch, test representative load, and monitor saturation	Batch metrics, target inventory
Pipeline auto-rollback makes outage worse	Rollback is unsafe for stateful change	Stop automation and execute tested recovery runbook	Rollback decision record, incident timeline
Production apply runs from shared runner	Weak environment protection	Use trusted isolated agents and environment approvals	Runner identity, environment policy

Hands-on Lab 45: Implement a Progressive Terraform-Ansible Delivery Pipeline

Prerequisites

- A disposable environment with at least three targets.
- Saved Terraform plan and built Ansible collection artifacts.
- Health checks and a simple verification script.

Implementation

1. Define change classes and required approvals for additive, replacement, and destructive changes.
2. Apply an approved saved plan using a protected delivery identity.
3. Refresh inventory and wait for explicit readiness checks.
4. Configure one canary target, verify health and telemetry, then expand to the next batch.

5. Inject a configuration failure and verify that promotion stops without reverting healthy infrastructure automatically.
6. Execute the documented recovery path and produce a complete change record.

Validation Evidence

- Only approved immutable artifacts are used.
- Promotion stops on failed verification.
- Target limits and batch sizes are visible in evidence.
- Recovery restores a known acceptable state.

Cleanup

- Destroy the test environment and revoke temporary identities.
- Retain the sanitized change record, approvals, and recovery evidence.

Interview Questions and Model Answers

1. Why can infrastructure rollback be dangerous?

Stateful or shared resources may have irreversible data or dependency changes, so automatic reversal can increase impact.

2. What should an infrastructure approver see?

Exact plan, risk, blast radius, policy and cost results, maintenance context, verification, and recovery decision.

3. How do you canary an infrastructure change?

Apply or configure a representative small subset, verify objective signals, then expand in controlled batches.

4. Why refresh inventory after apply?

Resource replacement or scaling can change the current target set even when the desired names remain similar.

Chapter Summary

- Delivery consumes approved artifacts; it does not rebuild them.
- Progressive change reduces blast radius.
- Verification controls promotion.
- Recovery must match the actual change semantics.

Chapter 46: Workload Identity, Secrets, and Keyless Automation

Automation credentials are high-impact assets because they can create infrastructure, change operating systems, retrieve secrets, and cross environment boundaries. Modern pipelines should use workload identity federation and short-lived credentials wherever possible, while ensuring that unavoidable secrets are centrally managed, narrowly scoped, rotated, and never embedded in state or logs.

Learning Objectives

- Design keyless CI/CD authentication.
- Separate Terraform, Ansible, verification, and break-glass identities.
- Prevent secret leakage through state, plans, variables, and logs.
- Implement rotation, revocation, and incident response for automation credentials.

Core Concepts

Workload identity federation: The CI/CD platform proves a signed workload identity to a cloud or automation service, which issues short-lived credentials based on repository, branch, environment, workflow, or audience claims. This removes long-lived cloud keys from secret stores.

Identity separation: Use distinct identities for plan, apply, inventory read, configuration, secret retrieval, verification, and break-glass operations. Separation limits blast radius and improves attribution.

Terraform secret exposure: Sensitive marks redact CLI display but do not remove values from state. Use provider-supported ephemeral or write-only arguments where available, external secret references, and minimal outputs. Encrypt and strictly control state storage.

Ansible secret exposure: Use Vault or an external secret manager for data at rest, `no_log` for carefully scoped tasks, and controller credential types for runtime injection. `no_log` is not a substitute for preventing secrets from entering unsafe variables or debug output.

SSH and host credentials: Prefer ephemeral certificates, short-lived SSH credentials, managed access services, or controller-injected keys. Shared static keys make revocation and attribution difficult.

Credential lifecycle: Every automation identity needs owner, purpose, scope, creation method, maximum lifetime, rotation, revocation, monitoring, and emergency disablement. Orphaned service identities are a common source of hidden privilege.

Reference Configuration or Command Pattern

```
# Example OIDC trust conditions (pseudocode)
issuer: https://token.actions.example
audience: cloud-provider
subject: repo:acme/platform:environment:production
claims:
  ref: refs/heads/main
  workflow: deploy.yml
credential_ttl: 900s
permissions:
  plan: read-only
  apply: environment-scoped
  ansible: host-group-scoped
```

Production Checklist

- CI/CD uses short-lived federated credentials where supported.
- Plan and apply identities are distinct and environment-scoped.
- No secret is committed, embedded in images, or exported in handoff artifacts.
- State, controller databases, logs, and caches are encrypted and access-audited.

- Credential revocation is tested and documented.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Federation works from any branch	Trust conditions omit repository or ref claims	Restrict subject, audience, workflow, and environment claims	Token claims, trust policy
Secret appears in Terraform state	Provider stores a configured value	Use external reference, ephemeral/write-only argument, or rotate after removal	State scan, provider schema
Ansible log reveals a password	Debug or failed task prints variable	Remove debug, scope no_log, and redesign secret flow	Redacted event log, task diff
Revoked key still works	Credential copied or cached elsewhere	Inventory all consumers and revoke at issuer and targets	Access logs, cache purge record

Hands-on Lab 46: Replace Static Pipeline Credentials with Federated Identity

Prerequisites

- A CI/CD platform capable of issuing OIDC or equivalent workload tokens.
- A disposable cloud account or local identity broker.
- Terraform and Ansible test workflows.

Implementation

1. Inventory every credential used by plan, apply, inventory, configuration, and verification stages.
2. Create separate short-lived identities and least-privilege policies for each stage.
3. Configure federation claims that restrict repository, branch, workflow, audience, and environment.
4. Remove static cloud keys and verify that the pipeline obtains temporary credentials only at runtime.
5. Scan state, artifacts, logs, and controller events for leaked secrets.
6. Revoke one identity during a test run and verify rapid failure, alerting, and recovery.

Validation Evidence

- No long-lived cloud key remains in the pipeline secret store.
- Temporary credentials expire automatically and are stage-specific.
- Secret scanning returns no unapproved values.
- Revocation produces attributable, bounded failure.

Cleanup

- Delete temporary trust relationships and test identities.
- Retain the redacted credential inventory and revocation evidence.

Interview Questions and Model Answers

1. Why is OIDC federation safer than static keys?

Credentials are short-lived and issued only when signed workload claims satisfy an explicit trust policy.

2. Does `Terraform sensitive=true` keep a value out of state?

No. It mainly redacts user-facing output; the value can still be stored in state unless an ephemeral or write-only mechanism is used.

3. When should `no_log` be used?

For narrowly scoped tasks that may expose secrets, while still designing the data flow to minimize secret presence and preserving non-sensitive diagnostics.

4. Why separate plan and apply identities?

Planning often needs broad read access but no mutation, while apply needs controlled write access only in the approved environment.

Chapter Summary

- Keyless short-lived identity is the preferred automation model.
- Sensitive display is not the same as secret absence.
- Identity separation limits blast radius.
- Revocation testing is part of credential engineering.

Chapter 47: Policy as Code, Compliance Gates, and Exception Management

Policy as code converts architectural, security, cost, reliability, and compliance expectations into repeatable tests. Effective policy systems evaluate structured change data, explain decisions, support severity and context, and manage exceptions as expiring governed objects rather than permanent bypasses.

Learning Objectives

- Design policy layers for Terraform plans and Ansible content.
- Use OPA/Rego or equivalent engines with structured inputs.
- Implement warnings, hard denies, and contextual approvals.
- Create auditable exception and waiver lifecycles.

Core Concepts

Policy layers: Static source policy catches prohibited patterns early. Plan policy evaluates intended infrastructure changes. Runtime and post-deployment policy verify the actual system. No single layer is complete.

Structured inputs: Terraform plan JSON, inventory schemas, Ansible lint data, collection manifests, SBOMs, and verification reports are more reliable than parsing console text. Normalize data before policy evaluation.

Decision model: Policies should return rule ID, severity, message, resource address, remediation, evidence, and documentation link. Binary pass/fail without explanation creates bypass pressure.

Context and ownership: The same change can have different risk in development and production. Context should be explicit and trusted: environment, data classification, service tier, exception ID, and owner.

Exceptions: An exception requires scope, rationale, compensating control, owner, approver, expiration, and review. It should be machine-readable and automatically fail after expiration.

Policy testing: Treat policy as software. Test allow and deny cases, edge conditions, schema changes, performance, and backward compatibility. Version policy bundles and record their digest with each decision.

Reference Configuration or Command Pattern

```
package terraform.security

import rego.v1

deny contains result if {
  rc := input.resource_changes[_]
  rc.type == "example_public_service"
  rc.change.after.public == true
  result := {
    "rule": "NET-001",
    "resource": rc.address,
    "message": "Public exposure requires an approved exception"
  }
}
```

Production Checklist

- Policies consume versioned structured schemas.
- Every decision is explainable and references a stable rule ID.
- Policy bundles have tests, owners, releases, and compatibility notes.
- Exceptions are scoped, approved, monitored, and expiring.
- Post-deployment verification confirms that intended controls exist at runtime.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Policy breaks after Terraform upgrade	Plan schema assumption changed	Pin versions and add schema compatibility tests	Tool version, failing input sample
Teams bypass policy routinely	Rules are noisy or lack remediation	Tune severity, improve explanations, and measure false positives	Exception trends, rule metrics
Expired waiver still permits deployment	Exception not evaluated as code	Add expiration enforcement and deny by default	Exception record, policy test
Source scan passes but runtime is noncompliant	Only pre-deployment layer exists	Add post-apply and continuous compliance checks	Runtime scan, corrective issue

Hands-on Lab 47: Build a Policy Bundle with Expiring Exceptions

Prerequisites

- Terraform plan JSON from a disposable environment.
- OPA or another policy engine.
- A sample Ansible repository and lint output.

Implementation

1. Define a normalized policy input schema for environment, service tier, plan changes, and exception metadata.
2. Implement rules for public exposure, encryption, required ownership, and prohibited Ansible shell usage.
3. Return structured decisions with rule IDs, resources, remediation, and severity.
4. Create unit tests for allowed, denied, malformed, and exception cases.
5. Add a time-bounded exception with compensating control and prove that it fails after expiration.
6. Record the policy bundle digest and decision report with the pipeline artifacts.

Validation Evidence

- Malformed or missing context fails closed.
- Policy explanations identify the exact resource or task.
- Expired exceptions are denied automatically.
- The same policy bundle digest can reproduce the decision.

Cleanup

- Remove test exception records and disposable resources.
- Retain policy tests and sanitized decision reports.

Interview Questions and Model Answers

1. Why evaluate Terraform plan JSON?

It exposes structured proposed changes, addresses, before/after values, and actions that policy can inspect deterministically.

2. What makes a good policy exception?

Narrow scope, clear rationale, compensating control, accountable owner and approver, expiration, and continuous visibility.

3. Why test policy like application code?

Policy defects can block all delivery or permit unsafe changes, so correctness and compatibility are production concerns.

4. Is pre-deployment policy sufficient?

No. Runtime drift, provider behavior, and out-of-band changes require post-deployment and continuous controls.

Chapter Summary

- Policy is a versioned software product.
- Structured inputs and explainable outputs reduce ambiguity.
- Exceptions are governed temporary objects.
- Continuous compliance complements pre-deployment gates.

Chapter 48: Software Supply-Chain Security, Provenance, and Artifact Trust

Infrastructure and configuration automation consume providers, modules, collections, roles, Python packages, container images, policies, and pipeline actions. A compromise in any dependency can obtain privileged execution. Supply-chain security therefore requires verified source, reproducible builds, provenance, trusted registries, scanning, signing, and enforcement at consumption time.

Learning Objectives

- Map the Terraform-Ansible automation supply chain.
- Generate and verify provenance and attestations.
- Sign and promote immutable artifacts.
- Enforce trust before plans, applies, and controller jobs.

Core Concepts

Supply-chain inventory: List every source repository, provider, module, collection, execution image, Python dependency, pipeline action, policy bundle, and base image. Record owner, version source, checksum, and update process.

Provenance: Provenance is verifiable information linking an artifact to source, builder, inputs, and build process. SLSA provides a progressive framework for increasing confidence that artifacts were built from the intended source and protected build system.

Signing and attestation: Sign modules, collection tarballs, container images, SBOMs, and provenance where tooling supports it. Verification must occur in the consumer path; producing signatures without enforcement adds little protection.

Trusted registries: Use approved private or mirrored registries with retention, immutability, vulnerability scanning, and access logs. Direct downloads from arbitrary URLs during production runs weaken reproducibility.

Dependency pinning: Terraform lock files, module versions, collection requirements, Python lock files, container digests, and pipeline action SHAs prevent silent upgrades. Automated update proposals should still pass full qualification.

Builder isolation: Trusted artifacts should be built in isolated ephemeral workers with minimal credentials. Build identities should publish artifacts but not deploy production, preserving separation of duties.

Reference Configuration or Command Pattern

```
# Example release manifest
release: automation-platform-2.4.0
source_commit: abcdef123456
artifacts:
  terraform_module:
    uri: registry.example/platform/network/2.4.0
    digest: sha256:...
  ansible_collection:
    uri: galaxy.example/acme/operations/3.1.0
    digest: sha256:...
  execution_environment:
    image: registry.example/ee/platform@sha256:...
attestations:
  provenance: provenance.intoto.jsonl
  sbom: sbom.spdx.json
```

Production Checklist

- All dependencies are pinned by version and, where possible, digest.
- Release artifacts are built by trusted isolated workers.
- Provenance and SBOMs are generated and retained.
- Consumers verify signatures, attestations, and approved registry origin.
- Compromised artifact revocation and replacement are tested.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Signed artifact is replaced without detection	Consumer does not verify digest or signature	Enforce verification and immutable registry tags	Verification log, registry event
Build cannot be reproduced	Unpinned base image or dependency	Pin all inputs and record builder environment	Manifest, lock files, build digest
Scanner passes source but image is vulnerable	Built artifact contains additional packages	Scan the final artifact and SBOM	Image scan, SBOM diff
Compromised version remains deployed	No revocation or inventory of consumers	Maintain deployment inventory and deny revoked digest	Consumer list, revocation record

Hands-on Lab 48: Publish and Enforce a Trusted Automation Release

Prerequisites

- A Terraform module, Ansible collection, and execution-environment image.
- A registry and signing/attestation tool such as Sigstore-compatible tooling.
- A CI environment with isolated build workers.

Implementation

1. Create a complete supply-chain dependency inventory.
2. Build the module package, collection tarball, and execution image from a protected commit.
3. Generate SBOM and provenance documents and record all digests.
4. Sign or attest the artifacts and publish them to an immutable approved registry.
5. Configure the delivery workflow to reject untrusted origin, invalid signature, or revoked digest.
6. Tamper with a test artifact and prove that consumption is blocked.

Validation Evidence

- Every deployed artifact maps to source, builder, inputs, and digest.
- Signature or attestation verification is mandatory.
- Tampered and revoked artifacts fail before execution.
- The release manifest can reconstruct the approved dependency set.

Cleanup

- Delete tampered test artifacts and revoke test signing identities.
- Retain sanitized provenance, SBOM, and verification evidence.

Interview Questions and Model Answers

1. What is provenance?

Verifiable metadata showing where an artifact came from, what inputs and builder produced it, and how it relates to source.

2. Why are signatures insufficient by themselves?

They provide value only when consumers verify them against trusted identities and enforce the result.

3. What should be pinned in an automation platform?

Terraform providers and modules, Ansible collections, Python packages, base images, pipeline actions, policy bundles, and execution images.

4. How does SLSA help?

It provides an incremental framework for improving source and build integrity and the trustworthiness of provenance.

Chapter Summary

- Automation dependencies are privileged software supply-chain inputs.
- Provenance links artifacts to trusted source and build.
- Verification must be enforced at consumption.
- Immutable registries and deployment inventory enable revocation.

Chapter 49: Integrated Testing, Ephemeral Environments, and Release Qualification

Terraform and Ansible must be tested together because isolated success does not prove that provisioned resources are reachable, correctly grouped, configurable, observable, or recoverable. Integrated qualification uses layered tests and disposable environments to verify the full lifecycle without risking production.

Learning Objectives

- Design a cross-tool test pyramid.
- Create and destroy ephemeral environments safely.
- Test idempotency, replacement, upgrade, and recovery.
- Define release qualification evidence and acceptance criteria.

Core Concepts

Test pyramid: Static checks and unit tests are broad and fast. Contract tests validate outputs, inventory, module and role interfaces. Integration tests create real disposable resources. End-to-end tests verify provision, configure, observe, recover, and destroy.

Ephemeral environment: A test environment needs unique naming, quotas, budget guardrails, TTL, owner metadata, isolated state, dedicated credentials, and guaranteed cleanup. Cleanup must run even when tests fail.

Lifecycle coverage: Test create, no-op repeat, update, replacement, scale, failure injection, configuration idempotency, restart, and destroy. Many defects appear only on the second or replacement run.

Representative conditions: Use realistic network controls, private access, DNS, identity, base images, data classification, and observability. An unrealistically permissive test environment produces false confidence.

Release matrix: Qualify supported Terraform versions, provider ranges, ansible-core versions, collection dependencies, operating systems, architectures, and execution environments. Define the minimum supported matrix and test it continuously.

Acceptance evidence: A release candidate should include tests, policy, plan summary, dependency and SBOM reports, idempotency, performance, recovery, and cleanup evidence linked to the source and artifact digests.

Reference Configuration or Command Pattern

```
test-run:
  id: pr-1842-7f3a
  ttl: 4h
  phases:
    - terraform_create
    - inventory_contract
    - ansible_converge
    - ansible_idempotence
    - failure_injection
    - recovery
    - terraform_destroy
  acceptance:
    unexpected_changes: 0
    leaked_resources: 0
    critical_policy_denies: 0
```

Production Checklist

- Ephemeral environments have unique state, TTL, owner, quota, and guaranteed cleanup.
- Tests cover repeat, update, replacement, failure, recovery, and destroy.
- Supported version matrices are explicit and automated.

- The environment resembles production security and connectivity.
- Release evidence links to immutable source and artifacts.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Test environment leaks resources	Cleanup depends on successful prior stages	Use finally/always cleanup and independent destroy credentials	Resource inventory, cleanup log
Integration passes but production fails	Test environment is overly permissive	Mirror identity, network, DNS, and policy constraints	Environment comparison
Second Ansible run changes files	Template nondeterminism or mutable data	Sort inputs, normalize timestamps, fix change detection	First/second diff
Provider upgrade breaks module	Compatibility matrix not tested	Add minimum/maximum and upgrade tests	Matrix report, lock diff

Hands-on Lab 49: Qualify a Full Terraform-Ansible Release in an Ephemeral Environment

Prerequisites

- Versioned Terraform module and Ansible collection artifacts.
- A CI environment with disposable cloud or local virtualization capacity.
- Policy, health, and cleanup verification scripts.

Implementation

1. Generate a unique test ID, isolated state backend key, TTL, budget, and owner metadata.
2. Provision the environment with Terraform and validate the inventory contract.
3. Run Ansible convergence twice and assert idempotency.
4. Upgrade or replace one resource, then verify configuration and service continuity.
5. Inject a failure, execute recovery, and confirm observability evidence.
6. Destroy the environment in an always-run cleanup stage and verify zero leaked resources.

Validation Evidence

- All lifecycle phases pass with linked evidence.
- Second convergence has no unexpected changes.
- Failure recovery meets the defined objective.
- Independent resource inventory confirms complete cleanup.

Cleanup

- Ensure the automatic cleanup completed and remove test credentials.
- Retain the release qualification report and artifact digests.

Interview Questions and Model Answers

1. Why is an end-to-end test necessary?

It validates the handoff and runtime interactions that module and role tests cannot see independently.

2. What is the most important cleanup control?

An independent always-run cleanup path with enough authority to destroy leaked resources even after earlier failures.

3. Why test replacement?

Replacement changes identities, addresses, inventory, readiness, and data behavior, revealing defects hidden by in-place updates.

4. What evidence qualifies a release?

Linked source and artifact digests plus test, policy, idempotency, performance, recovery, and cleanup results.

Chapter Summary

- Integrated behavior must be tested, not assumed.
- Ephemeral environments require strong cleanup engineering.
- Second runs and replacements reveal critical defects.
- Qualification evidence is a release artifact.

Chapter 50: Drift Detection, Reconciliation, and GitOps Operations

Drift is the difference between intended state and observed state. It may result from emergency action, manual change, provider behavior, failed automation, external controllers, or normal runtime processes. Mature operations classify drift before reconciling it and preserve the human decision that determined whether code or runtime should change.

Learning Objectives

- Detect Terraform and Ansible drift continuously.
- Classify intentional, emergency, external, and harmful drift.
- Design automated and approval-gated reconciliation.
- Prevent drift loops between multiple controllers.

Core Concepts

Terraform drift: Refresh-only plans and normal plans reveal differences between state, configuration, and provider-observed reality. Not every difference should be automatically applied; first identify whether the configuration, state, or runtime is authoritative.

Ansible drift: Scheduled check mode, compliance scans, file hashes, service state checks, or controller jobs can detect configuration differences. Check mode support varies by module, so critical controls may need explicit verification tasks.

Classification: Classify drift as approved emergency, unauthorized manual, external-controller owned, provider-computed, transient, or harmful. Classification determines whether to import, update code, ignore, reconcile, or investigate.

Reconciliation modes: Low-risk deterministic drift can be auto-remediated. Destructive, stateful, or ambiguous drift should create an issue or approval-gated run. A failed reconciliation must not loop indefinitely.

Competing reconcilers: Kubernetes operators, cloud autoscalers, security tools, Terraform, and Ansible can all update resources. Define ownership and ignore only the exact attributes controlled elsewhere.

Emergency changes: Break-glass changes must be time-bounded, attributable, monitored, and followed by a code reconciliation decision. The repository should not silently overwrite an emergency protection before review.

Reference Configuration or Command Pattern

```
# Drift workflow
terraform plan -refresh-only -detailed-exitcode -out=refresh.plan
terraform show -json refresh.plan > refresh.json
ansible-playbook compliance.yml --check --diff --limit production

# Classification
# 0 = no drift
# 1 = tooling error
# 2 = drift detected -> create governed reconciliation record
```

Production Checklist

- Drift jobs use read-only identities where possible.
- Terraform and Ansible drift are correlated by environment and change ID.
- Ownership and external-controller attributes are documented.
- Auto-remediation is limited to tested low-risk classes.
- Emergency changes create mandatory reconciliation records.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Drift job continually reports computed fields	Provider-managed value treated as desired input	Remove or ignore the exact provider-owned attribute	Plan diff, provider schema
Auto-remediation causes outage	Ambiguous or destructive drift class	Gate high-risk changes and require verification	Reconciliation policy, incident record
Ansible check mode reports false changes	Module lacks full check-mode support	Add explicit read-only verification tasks	Module attributes, verification output
Manual emergency control disappears	Reconciler overwrites before review	Pause affected policy or add time-bounded exception	Break-glass record, reconciliation commit

Hands-on Lab 50: Implement Classified Drift Detection and Reconciliation

Prerequisites

- Disposable Terraform-managed infrastructure.
- Ansible configuration and compliance playbook.
- Issue tracker or structured reconciliation record.

Implementation

1. Create scheduled read-only Terraform refresh and Ansible check workflows.
2. Introduce one unauthorized change, one external-controller change, and one approved emergency change.
3. Classify each drift item and record the authoritative owner and response.
4. Auto-remediate only the low-risk unauthorized change.
5. Create approval-gated reconciliation for the emergency change and update code or runtime deliberately.
6. Prove that the external-controller attribute no longer creates a drift loop.

Validation Evidence

- Every drift item receives a classification and owner.
- No destructive change is automatically applied.
- The emergency change is reconciled with code and evidence.
- Repeated drift scans converge without controller conflict.

Cleanup

- Restore the disposable environment and remove temporary exceptions.
- Retain sanitized drift and reconciliation records.

Interview Questions and Model Answers

1. Is all drift bad?

No. Some drift is intentional, emergency, provider-computed, or owned by another controller; it must be classified before action.

2. When is automatic reconciliation appropriate?

For deterministic, low-risk, tested changes with clear ownership and reliable verification.

3. Why can check mode be insufficient?

Module support varies and some predicted changes require real target information or custom read-only verification.

4. How should break-glass drift be handled?

Preserve it temporarily, record owner and expiration, and make an explicit decision to encode it or remove it.

Chapter Summary

- Drift detection is observation; reconciliation is a governed decision.
- Ownership prevents competing controller loops.
- Auto-remediation must be risk-limited.
- Emergency changes require follow-up in code.

Chapter 51: Automation Observability, SLOs, and Evidence Engineering

Automation is a production service. Teams need to know whether runs start, queue, authenticate, plan, apply, configure, verify, and clean up within expected time and risk limits. Observability combines logs, metrics, traces or correlation IDs, events, and evidence retention so operators can distinguish platform failure from target-system failure.

Learning Objectives

- Define telemetry for Terraform and Ansible delivery.
- Create SLOs for automation reliability and latency.
- Correlate pipeline, state, controller, target, and verification events.
- Design evidence retention without leaking sensitive data.

Core Concepts

Golden signals for automation: Measure success rate, queue time, execution latency, change volume, failure class, retry count, lock wait, inventory freshness, target reachability, idempotency, and cleanup leakage.

Correlation: Use one change or run ID across CI, HCP Terraform or backend logs, AWX/controller events, cloud audit logs, SSH certificates, application telemetry, and incident records. Correlation is more valuable than isolated verbose logs.

SLOs: Example SLOs include 99% of approved low-risk changes complete within 30 minutes, 99.9% of drift scans run on schedule, and zero unowned production applies. Error budgets can inform platform improvement versus feature work.

Structured events: Emit machine-readable stage, environment, artifact digest, actor, target count, result, duration, policy result, and recovery action. Human logs remain useful, but structured data enables reliable dashboards and alerts.

Evidence versus telemetry: Telemetry supports operations and may be high-volume. Evidence proves an approval, decision, artifact, change, or control. Define separate retention, immutability, and access requirements.

Sensitive data controls: Redact secrets, tokens, private keys, state values, host facts, and personal data. Limit high-verbosity logs in production and use controlled secure debug workflows when deeper diagnostics are required.

Reference Configuration or Command Pattern

```
{
  "change_id": "CHG-2026-0042",
  "stage": "ansible_configure",
  "environment": "production",
  "artifact_digest": "sha256:...",
  "targets": 24,
  "changed": 6,
  "failed": 0,
  "duration_seconds": 184,
  "policy_bundle": "sha256:...",
  "result": "success"
}
```

Production Checklist

- Every stage emits a shared correlation ID.

- Dashboards separate platform, dependency, target, and policy failures.
- SLOs are measurable and linked to operator action.
- Evidence artifacts are immutable and access-controlled.
- Logging and debugging controls prevent secret exposure.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Pipeline failed but cause is unknown	Logs lack stage and target correlation	Add structured events and one change ID	Cross-system event timeline
Alert volume is excessive	Every failed task pages operators	Alert on SLO impact and classified critical failures	Alert policy, noise trend
Audit evidence cannot reproduce change	Artifact digests or policy version missing	Record immutable references in every run	Evidence manifest
Debug logs leak secrets	High verbosity enabled broadly	Use secure time-bounded debug and redaction	Log policy, redaction test

Hands-on Lab 51: Build an Automation Observability and Evidence Dashboard

Prerequisites

- An integrated Terraform-Ansible test pipeline.
- A log or metrics platform, or local structured event collector.
- A sample evidence storage location with write-once simulation.

Implementation

1. Define event schema and one correlation ID propagated through every stage.
2. Emit structured events for plan, approval, apply, inventory, configure, verify, and cleanup.
3. Create metrics for success rate, duration, queue time, drift, idempotency, and leaked resources.
4. Define two SLOs and an error-budget policy for the automation platform.
5. Build a dashboard and alert that distinguish platform failure from target failure.
6. Create an immutable evidence manifest linking approvals, artifacts, policy, and verification.

Validation Evidence

- A single change can be reconstructed across all systems.
- Alerts trigger on SLO impact rather than individual harmless warnings.
- Evidence references immutable artifact and policy digests.
- Secret scanning of telemetry and evidence passes.

Cleanup

- Remove test alerts and sanitize retained events.
- Retain the dashboard definition, SLOs, and evidence manifest.

Interview Questions and Model Answers

1. What should an automation platform measure?

Reliability, queue and execution latency, change volume, failure class, locks, inventory freshness, idempotency, verification, and cleanup.

2. What is the difference between telemetry and evidence?

Telemetry supports operation and diagnosis; evidence proves a controlled decision or change and usually requires stronger retention and immutability.

3. Why use one correlation ID?

It enables a coherent timeline across CI, Terraform, Ansible, cloud audit, targets, and verification systems.

4. What is a useful SLO example?

A defined percentage of approved low-risk changes completing successfully within a target time, with exclusions and measurement clearly specified.

Chapter Summary

- Automation platforms need production-grade observability.
- Correlation turns distributed logs into an incident timeline.
- SLOs focus reliability investment.
- Evidence must be immutable, reproducible, and safe.

Chapter 52: Change Risk, Approvals, Rollback, and Incident-Safe Delivery

Infrastructure automation changes systems with different criticality, reversibility, and blast radius. A uniform approval process either blocks low-risk work or under-controls dangerous work. Risk-based delivery classifies changes, selects appropriate reviewers and maintenance controls, and prepares an incident-safe recovery path before execution.

Learning Objectives

- Create an objective infrastructure change-risk model.
- Map risk classes to approvals and delivery controls.
- Distinguish rollback, restore, recreate, failover, and roll-forward.
- Integrate change delivery with incident command.

Core Concepts

Risk factors: Consider environment, service tier, data classification, replacement or destroy actions, target count, shared dependencies, statefulness, novelty, time window, test coverage, drift, and recovery confidence.

Approval policy: Low-risk additive changes may be automatically approved after policy and tests. Medium-risk changes need service-owner review. High-risk or destructive changes need platform, security, data, or change-authority review and a maintenance window.

Four-eyes and separation: The person authoring a high-risk change should not be the sole approver or the identity that can bypass controls. Emergency paths should require separate break-glass authorization and post-event review.

Recovery vocabulary: Rollback reverts configuration. Restore recovers data or state. Recreate replaces infrastructure. Failover moves service. Roll-forward applies a corrective change. Use the correct term so operators understand the actual action.

Stop conditions: Define objective abort criteria: health degradation, error rate, failed target count, lock anomaly, unexpected destroy, policy mismatch, or evidence loss. Operators must be allowed to stop safely without being forced to complete the pipeline.

Incident integration: During an incident, change delivery should expose exact artifacts, affected resources, actor, timeline, and recovery options. Incident command owns coordination; automation operators execute bounded actions and preserve evidence.

Reference Configuration or Command Pattern

```

risk_score =
  environment_weight +
  destructive_actions * 5 +
  stateful_resources * 4 +
  shared_dependency * 3 +
  target_count_band +
  novelty +
  recovery_uncertainty

0-4    -> low: automated approval
5-9    -> medium: service owner
10-14  -> high: platform + owner
15+    -> critical: CAB/security/data + window

```

Production Checklist

- Risk is calculated from structured change data and documented context.
- Approval requirements scale with risk and service criticality.
- Recovery action is named and tested before execution.
- Abort criteria are objective and monitored.
- Emergency changes are attributable and receive post-incident review.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Every change waits for a committee	No risk differentiation	Automate low-risk classes and reserve review for material risk	Risk distribution, approval latency
Destructive plan receives low score	Model ignores plan actions or statefulness	Parse actions and add critical-resource weights	Plan JSON, score breakdown
Rollback runbook cannot restore data	Recovery term and method were wrong	Define restore or failover with tested RPO/RTO	Recovery test, data check
Operator continues despite degradation	No objective stop condition or authority	Add monitored abort gates and stop permission	Gate metrics, run timeline

Hands-on Lab 52: Implement Risk-Based Approval and Recovery Controls

Prerequisites

- Terraform plan JSON and Ansible change summary.
- A pipeline with environment approvals.
- A disposable service with health metrics.

Implementation

1. Define a risk model covering environment, actions, targets, statefulness, shared dependencies, novelty, and recovery confidence.
2. Calculate the score from structured plan and configuration data.
3. Map risk bands to reviewers, maintenance windows, and progressive rollout controls.
4. Create a high-risk replacement change and require the correct approval chain.
5. Define and test the actual recovery action, then inject a health degradation to trigger the abort gate.
6. Produce a change record showing score inputs, approvals, stop decision, and recovery result.

Validation Evidence

- The risk score is explainable and reproducible.
- Destructive stateful changes cannot bypass high-risk review.
- Abort criteria stop promotion automatically or by authorized operator.
- Recovery meets the declared objective.

Cleanup

- Restore the disposable service and remove temporary approval rules.
- Retain the sanitized risk and recovery evidence.

Interview Questions and Model Answers

1. Why use risk-based approvals?

They preserve speed for routine safe work while applying stronger review and controls to changes with real blast radius or uncertainty.

2. What is the difference between rollback and restore?

Rollback reverts configuration or release; restore recovers data or state from a protected copy.

3. What should trigger an abort?

Objective evidence such as health degradation, unexpected destructive action, failed target threshold, or policy mismatch.

4. How should emergency changes be governed?

With separate break-glass authorization, attribution, time bounds, monitoring, and mandatory post-event reconciliation and review.

Chapter Summary

- Approval depth should match measurable risk.
- Recovery language must match the actual mechanism.
- Abort gates protect against automation momentum.
- Incident command and delivery operations must integrate.

Chapter 53: Multi-Cloud Platform Factories and Self-Service Automation

Platform teams scale automation by turning infrastructure and configuration into supported products. A platform factory combines versioned Terraform modules, Ansible collections, policies, identity, observability, cost controls, documentation, and service ownership behind a self-service interface that produces compliant environments repeatedly.

Learning Objectives

- Design platform products rather than one-off pipelines.
- Create service catalog interfaces and golden paths.
- Support multiple clouds and environments without lowest-common-denominator design.
- Implement tenant isolation, quotas, ownership, and lifecycle management.

Core Concepts

Platform product: A product has users, owner, roadmap, support, SLOs, versioning, documentation, security model, telemetry, and deprecation process. A repository of modules without these properties is a component library, not a platform.

Golden path: A golden path is the easiest supported way to achieve a common outcome, such as a secure web service environment. It should encode default network, identity, logging, backup, policy, and configuration patterns while allowing governed extension.

Service catalog contract: The request interface should use business and operational inputs: service name, owner, tier, data classification, regions, capacity, and recovery objectives. It should not expose every provider knob.

Cloud-specific implementations: Keep a common platform contract where business intent is common, but allow cloud-specific modules and controls. Forcing all clouds into identical primitives often removes their strongest managed capabilities.

Tenant and environment isolation: Separate state, credentials, controller organizations, inventories, projects, quotas, and evidence by environment and tenant. Shared workers need strong workload isolation and short-lived identity.

Lifecycle: The factory must handle create, update, resize, rotate, recover, deprecate, and destroy. Day-two operations and ownership transfer are as important as initial provisioning.

Reference Configuration or Command Pattern

```
service_request:
  service: payments-api
  owner: team-payments
  tier: critical
  data_classification: confidential
  environment: production
  platform_product: secure-web-runtime
  version: 4.2.0
  regions: [primary, recovery]
  objectives:
    rto: 30m
    rpo: 5m
  capacity:
    profile: medium
```

Production Checklist

- Each platform product has owner, SLO, version policy, and support lifecycle.
- Self-service inputs express intent and validate ownership.
- Cloud-specific implementations satisfy one governed product contract.
- State, identity, quotas, inventory, and evidence are isolated appropriately.
- Destroy, transfer, recovery, and deprecation are supported workflows.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Catalog exposes hundreds of options	Provider schema leaked to users	Design intent-level profiles and governed extension points	Request schema, usability feedback
Product works only on creation	No day-two lifecycle workflows	Add update, resize, rotate, recover, and destroy journeys	Lifecycle test report
Multi-cloud module becomes lowest common denominator	Forced identical implementation	Keep common intent and separate cloud adapters	Architecture decision, capability matrix
Teams bypass the platform	Golden path is slower or too rigid	Measure adoption, simplify defaults, and provide extension contracts	Adoption metrics, bypass analysis

Hands-on Lab 53: Build a Self-Service Secure Runtime Platform Product

Prerequisites

- Terraform modules for at least one infrastructure target.
- Ansible collection for baseline and application configuration.
- A simple request form, YAML schema, or service catalog simulator.

Implementation

1. Define the product users, supported use case, owner, SLO, and lifecycle.
2. Create an intent-level request schema for ownership, tier, data, environment, capacity, and recovery objectives.
3. Map the request to versioned Terraform and Ansible artifacts with policy defaults.
4. Add isolated state, short-lived identities, dynamic inventory, observability, and evidence output.
5. Implement update and destroy journeys in addition to create.
6. Run two tenant requests and prove isolation, quota enforcement, and attributable ownership.

Validation Evidence

- The request schema rejects missing owner and invalid service tier.
- Both tenants receive isolated state, credentials, inventory, and evidence.
- Day-two update and destroy workflows succeed.
- Users do not need provider-specific implementation details.

Cleanup

- Destroy tenant test environments and revoke identities.
- Retain the product contract, lifecycle tests, and adoption metrics design.

Interview Questions and Model Answers

1. What makes an internal platform a product?

Named users and owner, supported outcomes, SLOs, roadmap, documentation, versioning, security, telemetry, support, and deprecation.

2. What is a golden path?

The easiest supported route to a common secure outcome, with sensible defaults and governed extension points.

3. How should multi-cloud abstraction work?

Share business intent and governance while allowing cloud-specific implementations and capabilities.

4. Why include destroy in the product lifecycle?

Unmanaged retirement creates cost, security, ownership, and data-retention risk; destruction must be as controlled as creation.

Chapter Summary

- Platforms package automation into supported products.
- Intent-level interfaces improve usability and governance.
- Multi-cloud should share contracts, not erase capabilities.
- Day-two lifecycle and ownership determine platform value.

Chapter 54: Audit, Compliance Evidence, Control Mapping, and Governance

Automation can reduce audit effort only when it produces trustworthy evidence. Compliance is not a collection of screenshots; it is a traceable chain from policy and control intent to source, review, artifact, execution, runtime verification, exception, and retained record. This chapter builds that evidence model.

Learning Objectives

- Map controls to automated preventive and detective mechanisms.
- Design evidence manifests and retention policies.
- Prove separation of duties and change traceability.
- Prepare automation systems for internal and external audits.

Core Concepts

Control mapping: Map each requirement to control objective, technical mechanism, owner, frequency, evidence, exception process, and test. One control may use policy, identity, workflow protection, runtime scan, and human review together.

Evidence chain: A strong chain links request, issue, commit, review, CI result, artifact digest, plan, policy decision, approval, apply, Ansible job, verification, cloud audit event, and final inventory. Missing links weaken attribution.

Preventive and detective controls: Branch protection and policy gates prevent some changes. Drift scans, audit logs, runtime compliance, and reconciliation detect what prevention missed. Both are necessary.

Separation of duties: Use repository reviews, environment approvals, distinct build and deploy identities, protected policy ownership, and logged break-glass access. Evidence should prove the actual enforcement configuration, not only the documented process.

Retention and privacy: Retention depends on regulation, investigation needs, and storage cost. Evidence stores require immutability, encryption, access control, legal hold, deletion policy, and minimization of secrets and personal data.

Continuous control monitoring: Automated controls should emit status and freshness. A policy that has not executed, a drift scan that is late, or a controller integration that is broken is itself a control failure.

Reference Configuration or Command Pattern

```
control_id: CM-03
objective: Production changes are authorized and traceable
preventive:
  - protected_branch
  - required_review
  - environment_approval
  - policy_gate
detective:
  - cloud_audit_log
  - drift_scan
  - evidence_chain_check
evidence:
  - commit_sha
  - review_ids
  - plan_digest
  - policy_digest
  - apply_run_id
  - verification_report
owner: platform-governance
frequency: continuous
```

Production Checklist

- Every control has owner, mechanism, evidence, frequency, and exception process.
- Evidence links are machine-generated and immutable.
- Separation-of-duties enforcement is exported and reviewed.
- Control freshness and execution failures are monitored.
- Retention and access comply with privacy and legal requirements.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Auditor receives screenshots only	No machine-readable evidence model	Generate manifests linking immutable system records	Evidence manifest, source links
Control is documented but not enforced	Process text mistaken for technical control	Export and test actual branch, IAM, and policy settings	Configuration export, control test
Evidence contains secrets	Unfiltered state or logs retained	Minimize, redact, encrypt, and restrict access	Secret scan, retention policy
Drift control silently stops running	No control freshness monitoring	Alert on missed schedules and stale evidence	Last-run metric, alert record

Hands-on Lab 54: Create an Audit-Ready Automated Control Evidence Package

Prerequisites

- A completed integrated pipeline run.
- Access to repository, CI, Terraform, Ansible/controller, and cloud audit records.
- A secure evidence storage location.

Implementation

1. Select three control objectives covering authorized change, least privilege, and configuration compliance.
2. Map each control to preventive, detective, owner, frequency, exception, and evidence fields.

3. Generate one evidence manifest that links the full change chain by immutable IDs and digests.
4. Export actual enforcement settings for branch protection, environment approval, and workload identity.
5. Create a control freshness dashboard and simulate a missed drift scan.
6. Run secret and privacy checks, then store the sanitized package immutably.

Validation Evidence

- An independent reviewer can reconstruct the change without operator explanation.
- Evidence proves actual enforcement, not only policy statements.
- Missed control execution triggers an alert.
- No unapproved secret or personal data appears in the package.

Cleanup

- Remove temporary exports containing sensitive details.
- Retain the sanitized evidence package and control mappings.

Interview Questions and Model Answers

1. What is a strong compliance evidence chain?

A linked sequence from request and reviewed source through artifacts, policy, approval, execution, runtime verification, and audit records.

2. Why are screenshots weak evidence?

They are hard to verify, search, reproduce, correlate, and prove immutable or complete.

3. What is control freshness?

Evidence that the control executed successfully within its required frequency and is still connected to the systems it protects.

4. Why need both preventive and detective controls?

Prevention cannot cover every path or failure, while detection identifies drift, bypass, and broken enforcement.

Chapter Summary

- Compliance value comes from traceable evidence, not document volume.
- Controls need owners, tests, and freshness monitoring.
- Actual enforcement configuration must be proven.
- Evidence retention must balance immutability, privacy, and access.

Chapter 55: Enterprise Integrated Automation Platform Capstone

The capstone combines the complete Volume 9E operating model into one production-style platform. Learners build a governed service factory that provisions infrastructure with Terraform, generates a validated handoff, configures systems with Ansible, applies policy and supply-chain controls, performs progressive delivery, observes the full lifecycle, reconciles drift, and produces audit-ready evidence.

Learning Objectives

- Design and implement the integrated platform architecture.
- Prove secure delivery, failure recovery, drift handling, and auditability.
- Operate the platform through realistic incidents and change scenarios.
- Produce portfolio-grade engineering and operational evidence.

Core Concepts

Reference architecture: The system includes Git repositories, artifact registries, CI workers, remote Terraform state or HCP workspaces, policy engine, workload identity, dynamic inventory, AWX/controller or Ansible runner, observability, evidence storage, service catalog, and target environments.

Product scope: Implement one supported platform product, such as a secure web runtime with network, compute, identity, logging, backup, baseline hardening, application deployment, health checks, and lifecycle operations.

Control plane security: Use protected source, isolated builders, signed artifacts, short-lived identities, least privilege, state protection, controller RBAC, immutable evidence, and break-glass governance.

Operational scenarios: The platform must handle normal create/update/destroy, failed apply, stale inventory, Ansible partial failure, policy denial, credential revocation, drift, emergency change, and recovery.

Acceptance gates: Success requires technical outcome, idempotency, progressive verification, security and compliance policy, observability, cleanup, recovery, evidence completeness, and a documented ownership model.

Portfolio narrative: Explain why architectural decisions were made, what trade-offs remain, how risk is controlled, and what evidence proves the system works. A polished repository without operational proof is incomplete.

Reference Configuration or Command Pattern

```
capstone acceptance:
  source_and_artifacts:
    protected_review: true
    immutable_versions: true
    provenance_verified: true
  identity:
    static_cloud_keys: 0
    stage_separation: true
  delivery:
    saved_plan_approved: true
    canary_verified: true
    ansible_idempotent: true
  operations:
    drift_classified: true
    recovery_tested: true
    evidence_complete: true
    leaked_resources: 0
```

Production Checklist

- Architecture and ownership are documented with decision records.
- All content and dependencies are versioned, pinned, and trusted.
- Identity is short-lived and separated by stage and environment.
- Policy, progressive delivery, observability, drift, and recovery are integrated.
- The final evidence package is complete, sanitized, and reproducible.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Capstone works only on happy path	Failure scenarios were not designed	Execute required injections and recovery tests	Incident timelines, recovery evidence
Evidence cannot map run to artifact	Missing correlation and digest fields	Use one change ID and immutable manifest	Evidence chain report
Platform is too broad to support	Product scope lacks clear boundary	Narrow to one golden path and documented extensions	Product contract, backlog
Cleanup leaves cost or security exposure	Destroy path not independently verified	Run leaked-resource inventory and forced cleanup	Cleanup report, zero-resource check

Hands-on Lab 55: Deliver and Defend an Enterprise Automation Platform

Prerequisites

- Completed Chapters 41-54 labs or equivalent components.
- Disposable infrastructure capacity and approved CI/CD environment.
- Git, artifact registry, Terraform backend, Ansible execution environment, policy engine, and observability stack.

Implementation

1. Define the platform product, users, owner, SLOs, supported lifecycle, threat model, and architecture decision records.
2. Build protected repositories, pinned dependencies, trusted artifacts, provenance, and federated identities.
3. Implement CI, plan and policy evidence, risk-based approval, apply, inventory contract, progressive Ansible configuration, and verification.
4. Add structured telemetry, evidence manifest, drift detection, control freshness, and self-service request interface.
5. Execute failure scenarios: stale inventory, partial Ansible failure, expired exception, credential revocation, emergency drift, and failed verification.
6. Recover each scenario, destroy the environment, verify zero leaked resources, and publish a sanitized portfolio report.

Validation Evidence

- The platform delivers the declared service outcome repeatedly.
- Second run is idempotent and replacement is handled safely.
- All mandatory failure scenarios have tested recovery.
- An independent reviewer can reconstruct every change and control.
- Final cleanup confirms zero unmanaged resources and credentials.

Cleanup

- Destroy all capstone resources and revoke signing, workload, controller, and break-glass identities.
- Archive only sanitized artifacts, evidence, diagrams, and runbooks.

Interview Questions and Model Answers

1. What is the hardest part of integrating Terraform and Ansible?

Defining ownership, handoff, failure recovery, and evidence across tools with different state and execution models.

2. How do you prove the platform is production-ready?

Through repeatable lifecycle tests, idempotency, failure injection, recovery, security and policy evidence, SLO telemetry, and complete cleanup.

3. What should be the capstone product boundary?

One well-supported golden path with explicit inputs, outputs, lifecycle, controls, owner, and extension points.

4. What distinguishes a strong portfolio submission?

Clear decisions and trade-offs, working artifacts, operational evidence, incidents and recovery, and an honest limitations roadmap.

Chapter Summary

- The capstone is an operating platform, not a sequence of scripts.
- Security, policy, evidence, and recovery are first-class capabilities.
- Failure scenarios prove engineering maturity.
- A complete lifecycle and ownership model define production readiness.

Part E Enterprise Portfolio Projects

Project	Portfolio Outcome
1. Keyless Multi-Stage Delivery	Implement separate federated identities for plan, apply, inventory, Ansible, and verification with revocation tests.
2. Terraform-Ansible Contract Service	Create a signed versioned inventory contract service with schema validation and freshness controls.
3. GitOps Drift Reconciler	Build classified drift detection that auto-remediates low risk and opens governed approvals for destructive changes.
4. Policy Bundle Platform	Publish tested OPA policies for infrastructure, configuration, cost, ownership, and exceptions.
5. Trusted Automation Supply Chain	Generate SBOMs, provenance, signatures, and mandatory verification for modules, collections, and execution images.
6. Ephemeral Qualification Factory	Create isolated test environments that prove create, repeat, replace, recover, and destroy.
7. Automation SLO Dashboard	Correlate CI, Terraform, Ansible, audit, and verification events into SLOs and evidence manifests.
8. Risk-Based Change Engine	Calculate plan and configuration risk, select approvers, and enforce progressive rollout and abort gates.
9. Multi-Cloud Service Factory	Expose one intent-level product contract backed by cloud-specific Terraform and Ansible implementations.
10. Continuous Compliance Evidence System	Map controls to preventive and detective automation with freshness monitoring and immutable evidence.

Senior Interview Drill Bank

How do you divide ownership between Terraform and Ansible?

Terraform owns provider-managed infrastructure lifecycle and state; Ansible owns configuration convergence. Each property must have one authoritative owner.

How do you safely pass Terraform results to Ansible?

Expose a minimal versioned output contract, validate it, build inventory from approved sources, refresh after apply, and retain the resolved target set.

What is the difference between CI and CD for infrastructure?

CI evaluates code and produces immutable evidence and artifacts without production mutation. CD consumes approved artifacts to make and verify controlled changes.

How do you avoid static credentials in pipelines?

Use workload identity federation with strict claims and short-lived environment-scoped credentials; separate identities by stage.

How do you manage policy exceptions?

As machine-readable, narrowly scoped, owner-approved, compensating-control-backed records with automatic expiration.

How do you secure the automation supply chain?

Pin dependencies, build in isolated workers, generate SBOM and provenance, sign artifacts, use trusted registries, and enforce verification.

What should happen after an Ansible partial failure?

Determine which targets changed, preserve evidence, fix the root cause, and resume idempotently with explicit limits rather than recreating infrastructure blindly.

How do you reconcile drift safely?

Detect read-only, classify ownership and risk, auto-remediate only proven low-risk drift, and approval-gate ambiguous or destructive changes.

What makes an automation platform observable?

Shared correlation IDs, structured stage events, SLOs, target and platform metrics, evidence manifests, and privacy-safe logs.

How do you prove compliance?

Link control intent to actual enforcement and an immutable evidence chain from source and review through artifacts, execution, runtime verification, and audit records.

When is rollback not the correct recovery?

For stateful, irreversible, or shared infrastructure changes where restore, recreate, failover, or roll-forward is the tested mechanism.

What defines a platform product?

Users, supported outcome, owner, SLO, versioning, security, lifecycle, documentation, telemetry, support, and deprecation.

Integrated Command and Investigation Reference

```
# Terraform automation
export TF_IN_AUTOMATION=1
terraform init -input=false
terraform fmt -check -recursive
terraform validate
terraform test
terraform plan -input=false -out=tfplan
terraform show -json tfplan > tfplan.json
terraform apply -input=false tfplan
terraform plan -refresh-only -detailed-exitcode

# Ansible automation
ansible-inventory -i inventory/ --graph
ansible-playbook site.yml --syntax-check
ansible-lint
ansible-playbook site.yml --check --diff --limit canary
ansible-playbook site.yml --serial 10% --limit production
ansible-navigator run site.yml --execution-environment true

# Policy and evidence
opa eval --data policy/ --input tfplan.json 'data.terraform.deny'
sha256sum tfplan tfplan.json *.tar.gz evidence-manifest.json
cosign verify <artifact>
cosign verify-attestation <artifact>
```

Production Readiness Checklist

- Ownership: one authoritative tool owns each mutable property; support and escalation are documented.
- Source: protected repositories, reviewed changes, immutable release references, and trusted workflow definitions.
- Dependencies: providers, modules, collections, Python packages, images, actions, and policies are pinned and inventoried.
- Identity: short-lived federated credentials, stage separation, least privilege, revocation, and break-glass controls.
- State and inventory: encrypted state, locking, backups, reduced handoff contract, freshness, and target evidence.
- CI: formatting, validation, lint, tests, policy, plan JSON, artifact packaging, and untrusted-change isolation.
- CD: exact artifacts, risk-based approval, protected environment, progressive rollout, verification, and abort gates.
- Secrets: no committed or embedded secrets; state, logs, controller events, and evidence are scanned and restricted.
- Policy: tested versioned bundles, explainable decisions, continuous checks, and expiring exceptions.
- Supply chain: trusted builders, SBOMs, provenance, signatures or attestations, approved registries, and verification.
- Testing: create, repeat, update, replacement, failure, recovery, destroy, compatibility matrix, and cleanup proof.
- Drift: scheduled detection, classification, owner mapping, limited auto-remediation, and emergency reconciliation.
- Observability: correlation IDs, structured events, SLOs, dashboards, alerts, and sensitive-data controls.
- Compliance: control mapping, actual enforcement exports, evidence chains, freshness monitoring, and retention policy.
- Lifecycle: create, update, resize, rotate, transfer, recover, deprecate, and destroy are supported and documented.

Glossary

Term	Definition
Artifact promotion	Moving the same immutable approved artifact through environments rather than rebuilding it.
Attestation	A signed statement about an artifact, such as provenance, test result, or policy result.
Break-glass	Emergency privileged access that is separately authorized, monitored, time-bounded, and reviewed.
Change correlation ID	Identifier propagated across source, pipeline, Terraform, Ansible, audit, verification, and incident records.
Continuous reconciliation	Repeated comparison and controlled correction of observed state against declared desired state.
Control freshness	Evidence that a control executed successfully within its required interval and remains connected.

Term	Definition
Drift	Difference between intended, recorded, and observed state.
Ephemeral environment	Short-lived isolated environment created for a test or change and automatically destroyed.
Evidence manifest	Machine-readable index linking approvals, artifacts, policy, execution, verification, and audit records.
Golden path	The easiest supported and governed route to a common platform outcome.
GitOps	Operation of systems through declarative, versioned desired state and controlled continuous reconciliation.
Handoff contract	Versioned interface carrying stable outputs and metadata between automation stages.
Idempotency	Repeated execution converges to the same desired state without unintended additional changes.
Policy as code	Machine-evaluated rules versioned, tested, and released like software.
Provenance	Verifiable information linking an artifact to source, builder, inputs, and build process.
Reconciliation	Governed decision and action to resolve drift by changing code, state, or runtime.
Recovery objective	Defined target for restoration time, data loss, or service state after failure.
SLO	Measurable target for reliability or performance of a service or automation platform.
Supply chain	All sources, dependencies, builders, registries, artifacts, and consumers involved in automation delivery.
Workload identity federation	Exchange of trusted workload claims for short-lived credentials without storing long-lived keys.

Official Documentation and Standards Index

Topic	Official Source
Terraform automation	https://developer.hashicorp.com/terraform/tutorials/automation/automate-terraform
Terraform plan and apply	https://developer.hashicorp.com/terraform/cli/commands/plan
Terraform machine-readable plan JSON	https://developer.hashicorp.com/terraform/internals/json-format

Topic	Official Source
Terraform dependency lock file	https://developer.hashicorp.com/terraform/language/files/dependency-lock
HCP Terraform run workflow	https://developer.hashicorp.com/terraform/cloud-docs/workspaces/run
HCP Terraform actions	https://developer.hashicorp.com/terraform/cloud-docs/actions
Ansible inventory plugins	https://docs.ansible.com/projects/ansible-core/devel/plugins/inventory.html
Ansible dynamic inventory	https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_dynamic_inventory.html
AWX inventories and Terraform state source	https://docs.ansible.com/projects/awx/en/latest/userguide/inventories.html
Ansible check and diff mode	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_checkmode.html
Ansible Vault	https://docs.ansible.com/projects/ansible/latest/vault_guide/index.html
OpenGitOps principles	https://opengitops.dev/
Open Policy Agent Terraform guidance	https://www.openpolicyagent.org/docs/terraform
SLSA specification	https://slsa.dev/spec/v1.2/
SLSA provenance	https://slsa.dev/spec/v1.2/provenance
NIST SP 800-218 SSDF	https://csrc.nist.gov/pubs/sp/800/218/final

Technology and certification details change. Before implementing production controls, verify the current official documentation, supported versions, provider and collection compatibility, and organizational policy requirements.